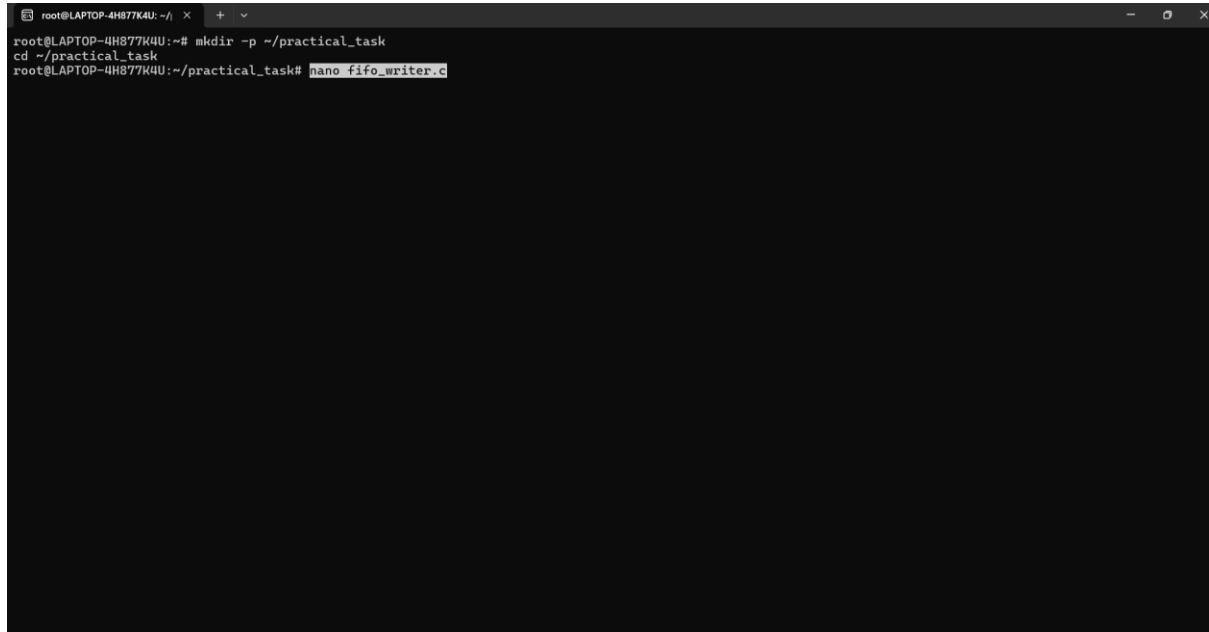


Practical – 6

Fifo_writer:

A terminal window with a dark background. The prompt is 'root@LAPTOP-4H877K4U: ~/'. The user enters 'mkdir -p ~/practical_task', followed by 'cd ~/practical_task', and finally 'nano fifo_writer.c'. The file name 'fifo_writer.c' is highlighted in yellow in the last command.

```
root@LAPTOP-4H877K4U: ~/   
root@LAPTOP-4H877K4U:~# mkdir -p ~/practical_task  
cd ~/practical_task  
root@LAPTOP-4H877K4U:~/practical_task# nano fifo_writer.c
```

Code:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <sys/stat.h>  
  
#define FIFO_FILE "/tmp/myfifo"  
  
int main()  
{  
    char message[100];  
  
    // Create named pipe  
    if (mkfifo(FIFO_FILE, 0666) == -1)
```

```

{
    perror("mkfifo");
}

printf("FIFO Writer\n");
printf("Enter a message: ");

fgets(message, sizeof(message), stdin);

// Open FIFO for writing
int fd = open(FIFO_FILE, O_WRONLY);

if (fd == -1)
{
    perror("open");
    return 1;
}

write(fd, message, strlen(message) + 1);

printf("Message sent successfully!\n");

close(fd);

return 0;
}

```

Fifo_reader:

```

#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

```

```
#include <unistd.h>

#include <sys/stat.h>

#define FIFO_FILE "/tmp/myfifo"

int main()
{
    char message[100];

    // Create FIFO if it does not exist
    mkfifo(FIFO_FILE, 0666);

    printf("FIFO Reader\n");
    printf("Waiting for message...\n");

    // Open FIFO for reading
    int fd = open(FIFO_FILE, O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    read(fd, message, sizeof(message));
```

```
printf("Message received: %s\n", message);

close(fd);

unlink(FIFO_FILE);

return 0;
}
```

Compilation and output:

```
root@LAPTOP-4H877K4U: ~/ | x + v
root@LAPTOP-4H877K4U:~# mkdir -p ~/practical_task
cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano fifo_writer.c
root@LAPTOP-4H877K4U:~/practical_task# nano fifo_reader.c
root@LAPTOP-4H877K4U:~/practical_task# gcc fifo_writer.c -o fifo_writer
root@LAPTOP-4H877K4U:~/practical_task# gcc fifo_reader.c -o fifo_reader
root@LAPTOP-4H877K4U:~/practical_task# ls
client client.c fifo_reader fifo_reader.c fifo_writer fifo_writer.c server server.c server.c signals signals.c
root@LAPTOP-4H877K4U:~/practical_task#
```

Terminals

```
root@LAPTOP-4H877K4U: ~/ | x + v
root@LAPTOP-4H877K4U:~# cd ~/practical_task
./fifo_reader
FIFO Reader
Waiting for message...
Message received: Hello from Writer
root@LAPTOP-4H877K4U:~/practical_task#

root@LAPTOP-4H877K4U: ~/ | x + v
root@LAPTOP-4H877K4U:~# cd ~/practical_task
./fifo_writer
mkfifo: File exists
FIFO Writer
Enter a message: Hello from Writer
Message sent successfully!
root@LAPTOP-4H877K4U:~/practical_task#
```

PART B — COMMON SIGNALS

Create `signals.c`

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <sys/wait.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received - Interrupt signal (Ctrl+C)\n");
    }

    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received - Termination signal\n");
    }

    else if (sig == SIGCHLD)
    {
        printf("\nSIGCHLD received - Child process terminated\n");
    }
}
```

```
// Prevent zombie process
while (waitpid(-1, NULL, WNOHANG) > 0);
}

else if (sig == SIGSEGV)
{
    printf("\nSIGSEGV received - Segmentation fault\n");
    printf("Program will terminate safely.\n");

    _exit(1);
}

else if (sig == SIGPIPE)
{
    printf("\nSIGPIPE received - Broken pipe\n");
}

fflush(stdout);
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
```

```
signal(SIGCHLD, signal_handler);
signal(SIGSEGV, signal_handler);
signal(SIGPIPE, signal_handler);

printf("=====\n");
printf("    COMMON SIGNAL DEMO\n");
printf("=====\n");

printf("Process ID: %d\n", getpid());

printf("\nSignals handled:\n");
printf("SIGINT - Ctrl+C\n");
printf("SIGTERM - Termination\n");
printf("SIGCHLD - Child termination\n");
printf("SIGSEGV - Segmentation fault\n");
printf("SIGPIPE - Broken pipe\n");

printf("\nWaiting for signals...\n");

while (1)
{
    pause();
}

return 0;
```

```
}
```

Compilation and output:

```
root@LAPTOP-4H877K4U: ~/practical_task
nano signals.c
root@LAPTOP-4H877K4U:~/practical_task# gcc signals.c -o signals
root@LAPTOP-4H877K4U:~/practical_task# ls
client client.c fifo_reader fifo_reader.c fifo_writer fifo_writer.c server server.c server.c signals signals.c
root@LAPTOP-4H877K4U:~/practical_task# ./signals
=====
      POSIX SIGNAL HANDLING
=====
Process ID (PID): 1863
Waiting for signals...
|
```